

Summer School · EIV 2026

GNNs & LLMs: Reasoning Beyond Text

Dr. Daniel F. Pérez-Ramírez

Guest at Escuela Colombiana de Ingeniería Julio Garavito
2026-06-30

5 days · 24 hrs · In-Person



SECTION 1

Course Kick-Off

About You

Course Expectations & Your Background

Go to

www.menti.com

Enter the code

2440 4000



Or use QR code

About Me

Professional Experience

- **04/2026 – today** Senior Data Scientist, **Nordea Bank Abp**, Stockholm (SWE)
- **10/2019 – 03/2026** Machine Learning Researcher, **RISE Research Institutes of Sweden**, Stockholm (SWE)
- **08/2019 – 10/2018** Algorithm Developer, **Mercedes Benz AG**, Stuttgart (DEU)
- **09/2017 – 09/2018** Research Assistant, **fortiss GmbH**, Munich (DEU)
- **03/2017 – 08/2017** Research Assistant, **TUMCREATE Ltd.**, Singapore (SGP)

Education

- **03/2026** Ph.D. Computer Science, **KTH Royal Institute of Technology**, Stockholm (SWE)
- **07/2019** M.Sc. Robotics, Cognition, Intelligence, **TUM Technische Universität München**, Munich (DEU)
- **09/2016** B.Sc. Mechanical Engineering, **TUM Technische Universität München**, Munich (DEU)

How We Will Work

Intuition over Formalism

More important to understand principles than formulas

Learn by Doing

Labs build the low-level understanding of the methods

Project to explore and be creative to apply the methods learnt

ALWAYS interrupt

If you have questions

ALWAYS speak up

if you don't understand something

Go Beyond Scope

Open to discuss topics beyond the course contents.

Course Overview

I · 5 hrs

ML Foundations

ML best practices. Train first ML model.

II · 5-6 hrs

LLMs + RAG

Fundamentals of LLMs and first RAG system.

III · 5-6 hrs

GNNs

Encode relational structure.

IV · 5-6 hrs

LLMs + GNNs

Combine all three end-to-end

V · 3-4 hrs

Wrap-up + Present

Demo your course project.

Course Objectives

O1: Build a solid understanding of machine learning and ML model training-deployment.

O2: Understand the inner-working of LLMs and the cause of their limitations.

O3: Understand GNNs, their applicability, and the most popular architectures.

O4: First PoC of combining LLMs and GNNs for advanced reasoning.

SECTION 2

Why Reasoning Beyond Text?

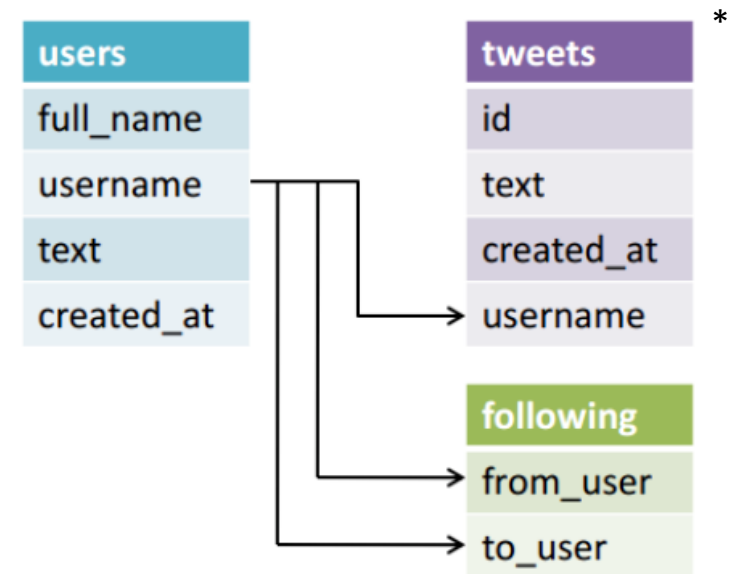
Why Reasoning Beyond Text?

WHAT LLMs DO WELL



- ✓ Model language distribution by next-word prediction with context
- ✓ Powerful multimodal models

WHAT THEY MISS



Relational Structure of the Data

* Image source: <https://code.tutsplus.com/relational-databases-for-dummies--net-30244t>

Hallucination

PROMPT

"Which country hosted the 1916 Summer Olympics?"

CONFIDENT ANSWER

"Germany — Berlin hosted the 1916 Summer Olympics."

REALITY

The 1916 Games were cancelled — World War I. They never took place.

Plausible-sounding, confidently wrong.

The model cannot distinguish **what it knows** from **what it infers**.

**Hallucinations are not bugs.
They are structural.**

Why These Models for the Course?

LLMs



Almost everywhere now

GNNs



Niche application areas

Financial systems

Social networks

Healthcare

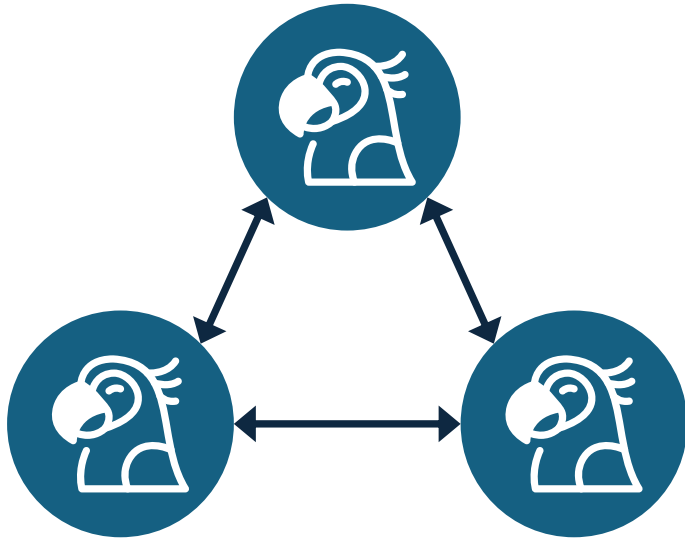
Supply chain

E-commerce

Real-world data is relational

Possible Future Direction for LLMs

Multi-Agent Systems



RecursiveMAS

Recursive Multi-Agent Systems

Xiyuan Yang^{1,*}, Jiaru Zou^{1,2,*†}, Rui Pan¹, Ruizhong Qiu¹, Pan Lu², Shizhe Diao³, Jindong Jiang³, Hanghang Tong¹, Tong Zhang¹, Markus J. Buehler⁴, Jingrui He¹✉, James Zou²✉

¹UIUC ²Stanford University ³NVIDIA ⁴MIT

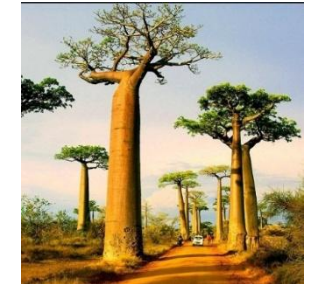
Fresh off the oven: **26th April 2026**

3 small LLMs achieve similar performance to proprietary models

SECTION 3

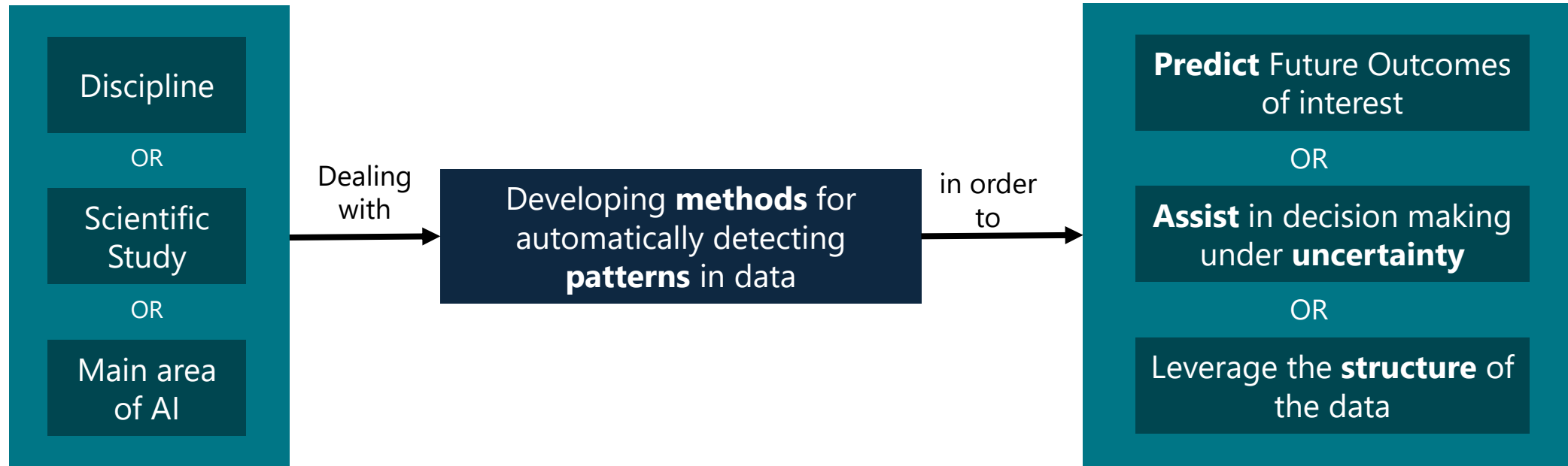
Machine Learning Fundamentals

Sacha



P A T T E R N S

What is Machine Learning?



From now on, Machine Learning → **ML**

What is Machine Learning?

RULE-BASED SYSTEMS

A human writes the if/then logic based on experience.
Humans are a “filtering stage” for **better** or for **worse**.



```
if (amount > $10k):  
    flag_fraud()  
else:  
    no_fraud()
```

Model structured defined by fixed rules

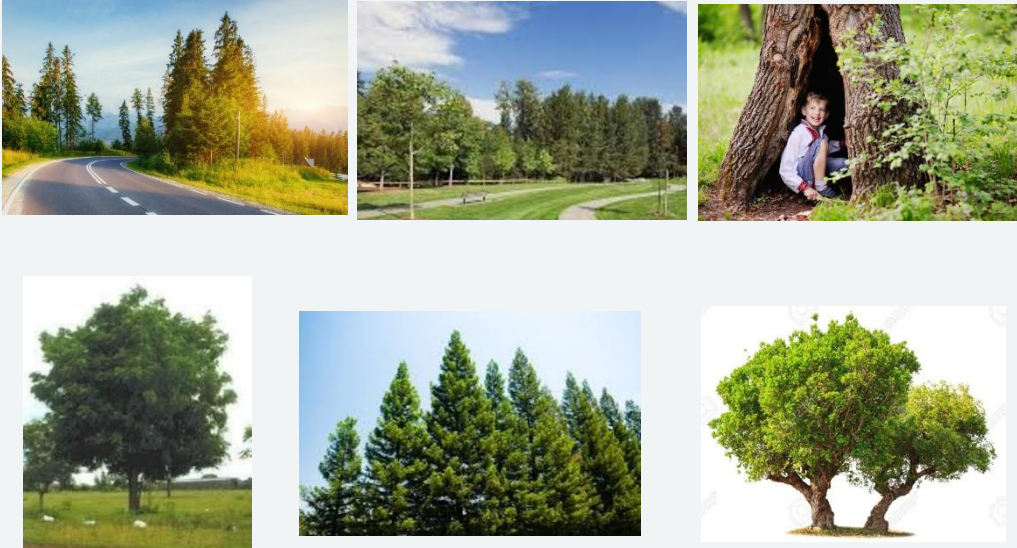
MACHINE LEARNING

Show labeled examples; the algorithm infers the pattern.
Adapts and scales.



Model structured defined probabilistically

ML Model Generalization



TRAINING

Find the model **structure** and **hyper-parameters**.

GENERALIZATION



DEPLOYMENT

Inference on **previously unseen** data.

ML has Many “Flavors”

Supervised

Labeled Examples → Predict labels

Classification

Regression

Ranking

Unsupervised

Find structure in unlabeled data

Clustering

Dimensionality
reduction

(some) Anomaly
Detection

Reinforcement

Agent learns by trial and error

Multi-armed
bandits

Policy-based

Value-based

Multi-agent RL

Semi-Supervised

Train with a **small** sample of **labelled** + large sample of unlabeled data.

Label Propagation
(graph)

Pseudo-labeling

Self-Supervised

Use **unlabeled** data and hide part of the structure to predict the structure.

Masked token
prediction

Contrastive
Learning

Probability Theory Refresher

Many ML training data is i.i.d.

- Probability is non-negative.
- Must add up to 1 across all possible values of the random variables

I.I.D.

Independently and Identically Distributed random variables.

Independent

A and B are independent if:

$$\mathbb{P}(A, B) = \mathbb{P}(A) \mathbb{P}(B)$$

$$\mathbb{P}(A \mid B) = \mathbb{P}(A)$$

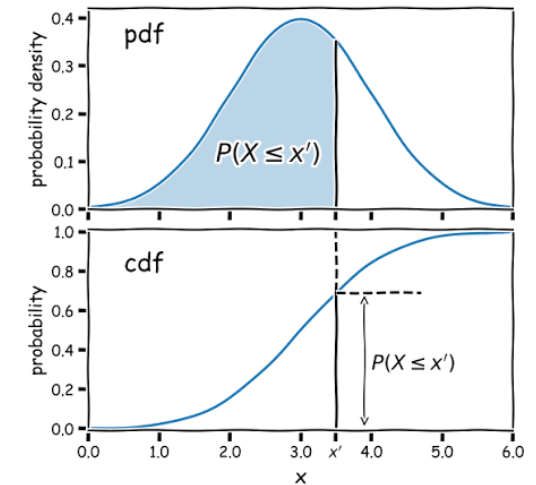
A and B are independent given C:

$$\mathbb{P}(A, B \mid C) = \mathbb{P}(A \mid C) \mathbb{P}(B \mid C)$$

Identical

X and Y are identical, if they have the same Cumulative Distribution Function (**CDF**)

$$F_X(x) = F_Y(x)$$



Probability Theory Refresher

- Probability is non-negative.
- Must add up to 1 across all possible values of the random variables

Sum Rule

Marginal Joint

$$p(X) = \sum_Y p(X, Y)$$

Product Rule

$$p(X, Y) = p(Y|X)p(X)$$

Bayes Rule

$$p(Y|X) = \frac{p(X|Y)p(Y)}{p(X)}$$

Probability Theory Refresher

Expected Value

Probability-weighted average

A.k.a. "mean" or "average"

$$\mathbb{E}[X] = \sum_x x p(x)$$

Variance

Expected squared deviation from the mean

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

$$\text{Var}(X) = \text{Cov}(X, X)$$

Std.-Dev.

$$\sigma = \sqrt{\text{Var}(X)}$$

Covariance

Expected product of two variables' deviations from their means.
Measures how they vary together.

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$$

$$\text{Cov}(X, Y) = \text{Cov}(Y, X)$$

Scale Dependent

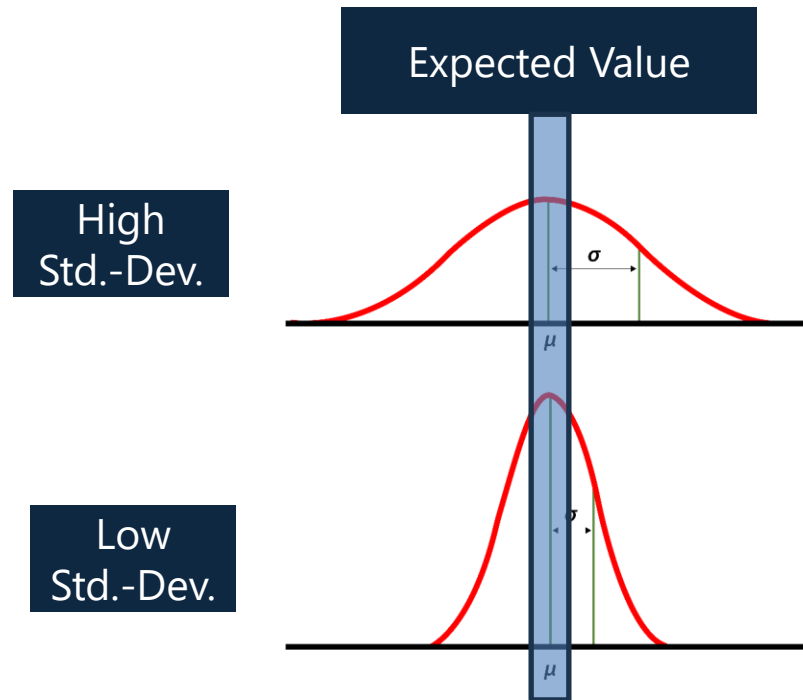
Correlation coeff.

Normalized, dimensionless measure of the strength and direction of the linear relationship

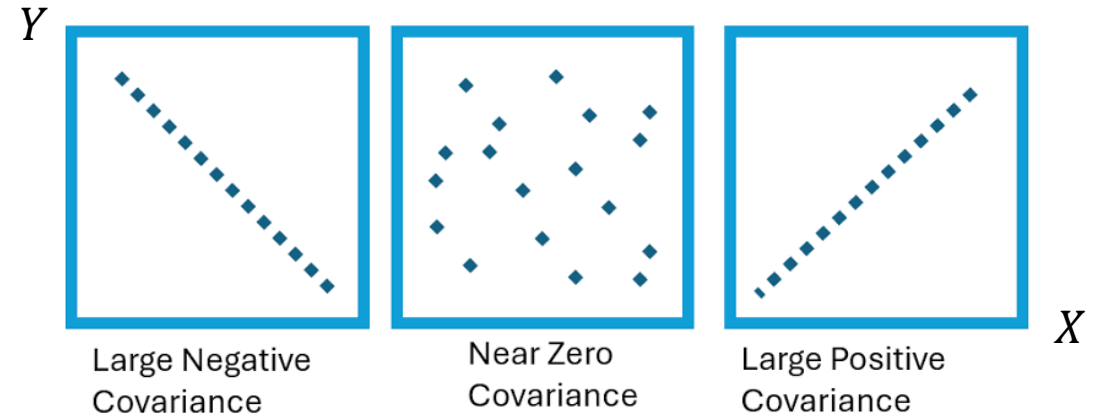
$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Bounded [-1,+1]

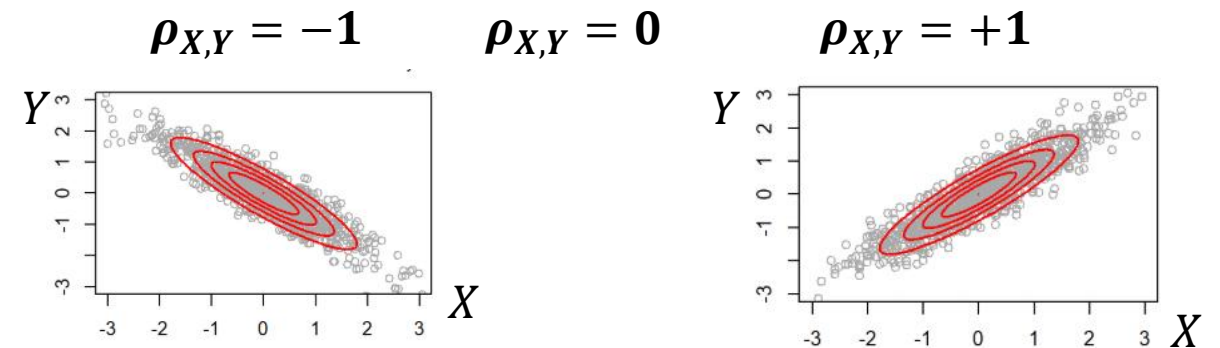
Probability Theory Refresher



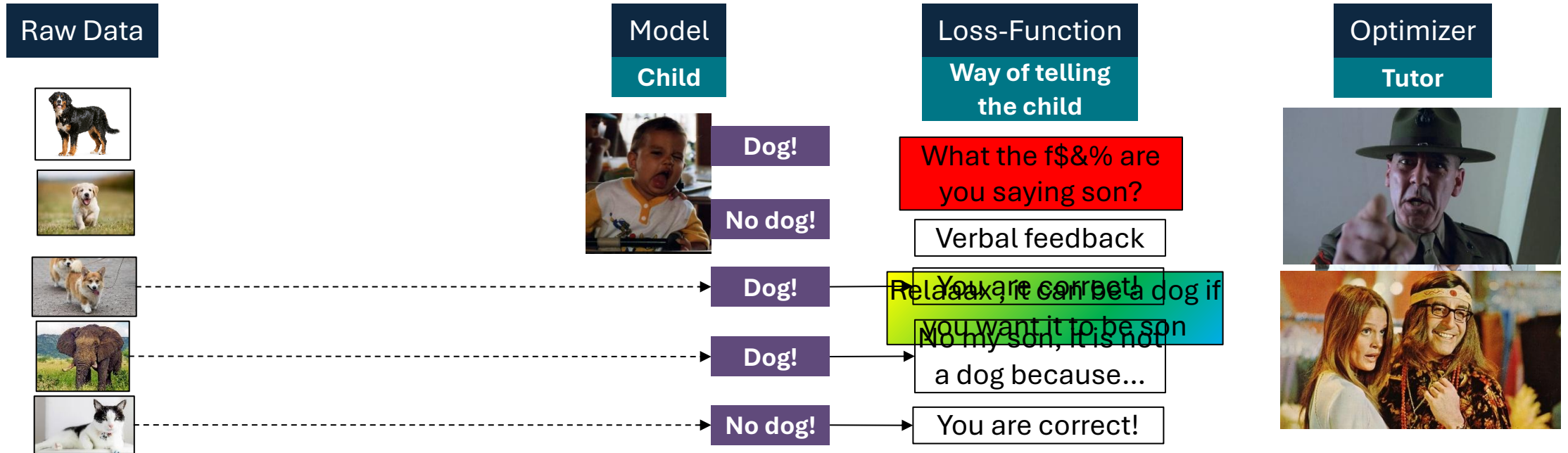
Covariance



Correlation coeff.



Supervised Learning Fundamentals



Types of ML Models: Generative vs Discriminative

GENERATIVE

Learn the **joint probability distribution** of the input data x and (if existent) output data y .

$$\mathcal{D} = \{x_i\}_{i=0}^N \longrightarrow P(x)$$

$$\mathcal{D} = \{(x_i, y_i)\}_{i=0}^N \longrightarrow P(x, y)$$

Allows to generate new samples or model missing data

DISCRIMINATIVE

Learn the **conditional distribution** of the input data x and output data y .

$$\mathcal{D} = \{(x_i, y_i)\}_{i=0}^N \longrightarrow P(y|x)$$

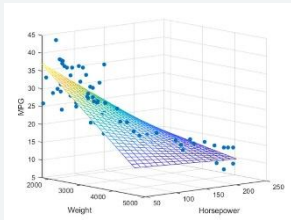
Directly learn the input-output relationship.

Types of ML Models: Parametric vs Non-Parametric

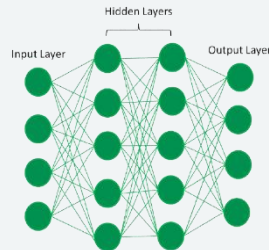
PARAMETRIC

Fixed functional form with a set of parameters Θ

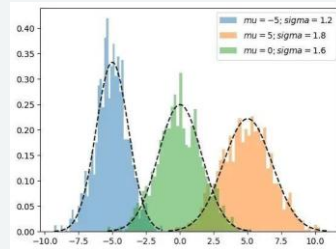
Linear
regression



Deep
Learning



Gaussian
Mixture

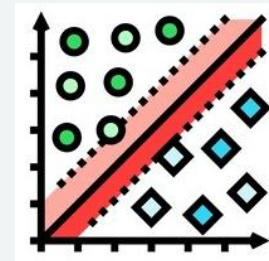


Complexity grows with size of Θ .

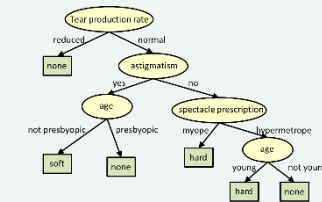
NON-PARAMETRIC

Not defined by parameters.

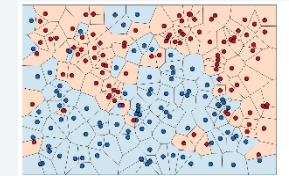
SVMs



Tree-based
models



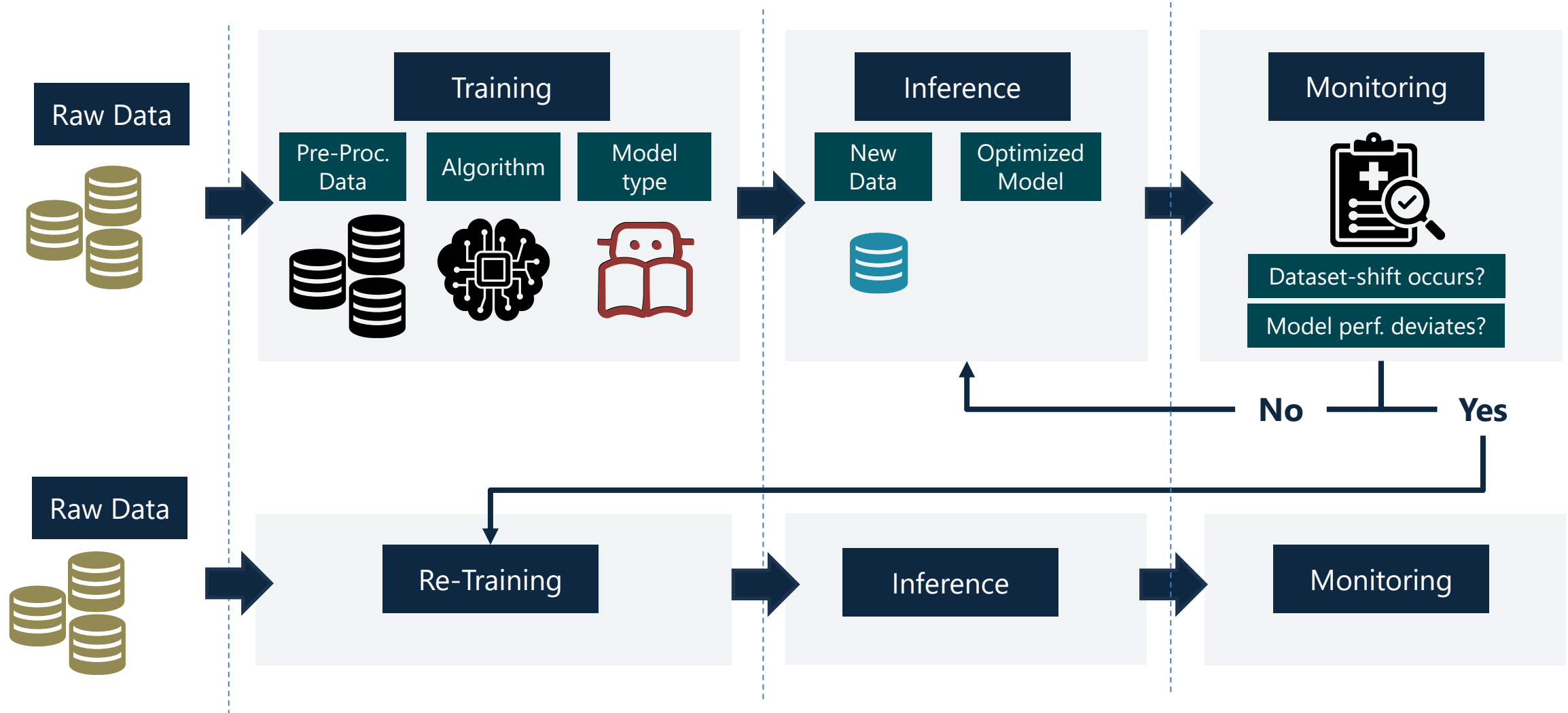
k-NN



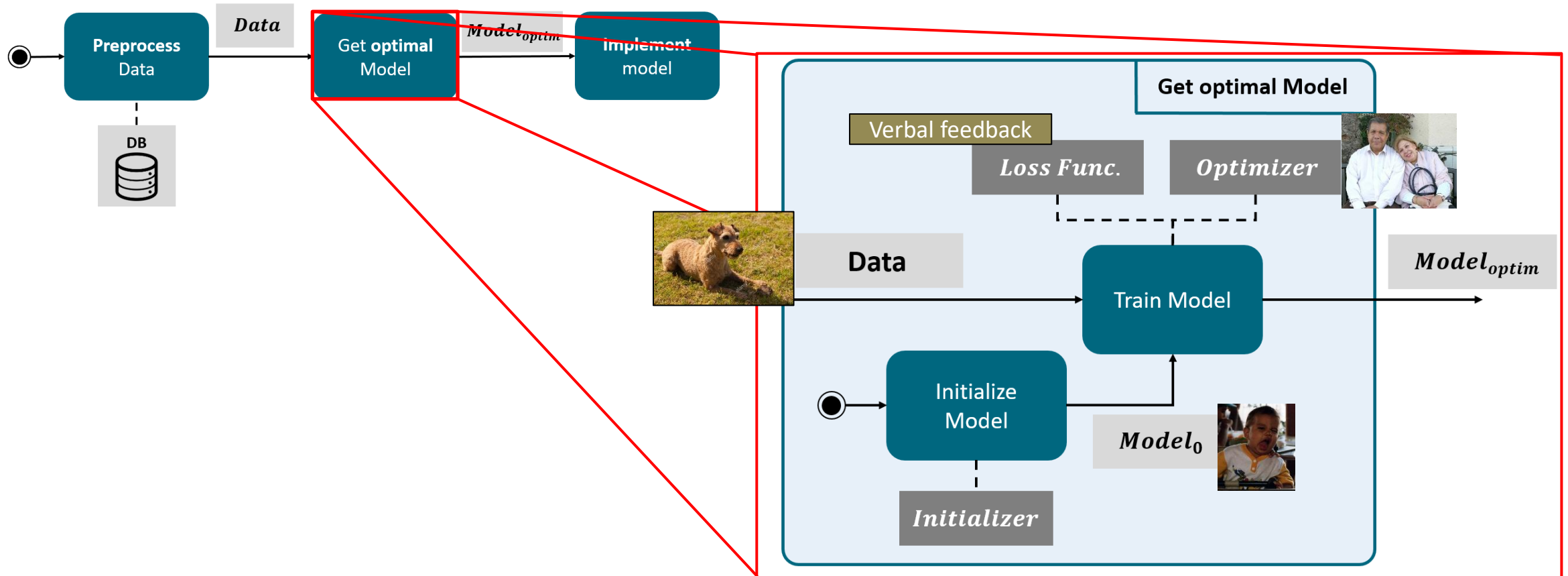
Complexity proportional to data samples.

Both parametric and non-parametric have hyper-parameters

Machine Learning Pipeline

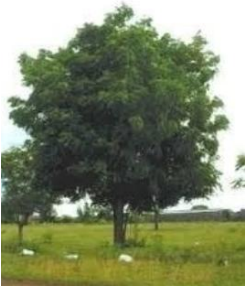


Supervised Learning Fundamentals



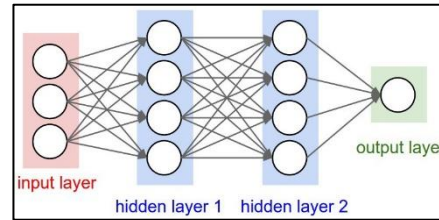
Supervised Learning Fundamentals

Raw Data



$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

Model



$$f_{\theta}: \mathcal{X} \rightarrow \mathcal{Y}$$

Loss-Function

Evaluate if model prediction is good or not

Optimizer

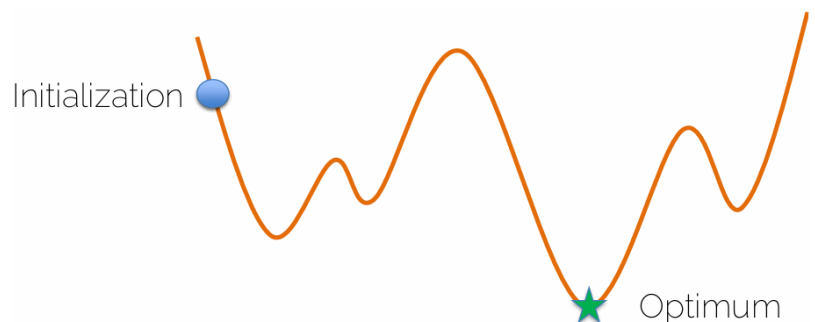
Uses the loss to **modify** model weights

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(x), y)]$$

Gradient Descent

Goal

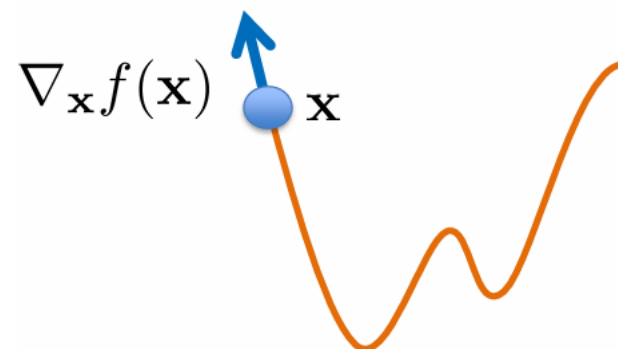
$$\mathbf{x}^* = \arg \min f(\mathbf{x})$$



Find parameters $\mathbf{x}^*$ so that our loss f is minimized

Derivative to gradient

$$\frac{df(x)}{dx} \longrightarrow \nabla_{\mathbf{x}} f(\mathbf{x})$$

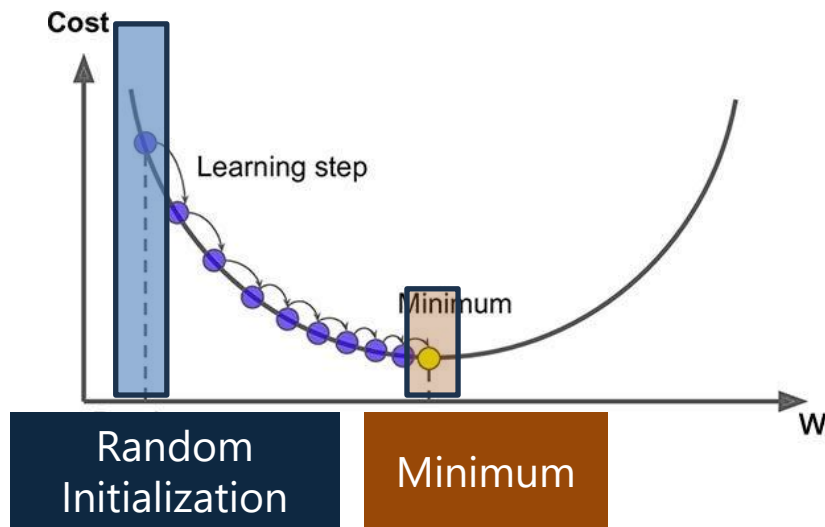


Make $\mathbf{x}$ follow the negative gradient direction:

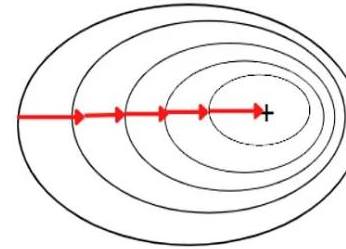
$$\mathbf{x}' = \mathbf{x} - \epsilon \nabla_{\mathbf{x}} f(\mathbf{x})$$

Learning rate

Optimization of a Parametrized ML model: Gradient Descent (GD)

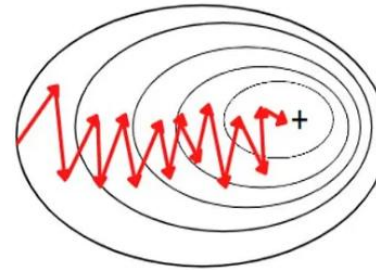


Batch GD



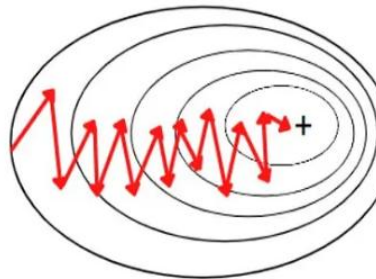
One update using the whole dataset

Stochastic GD



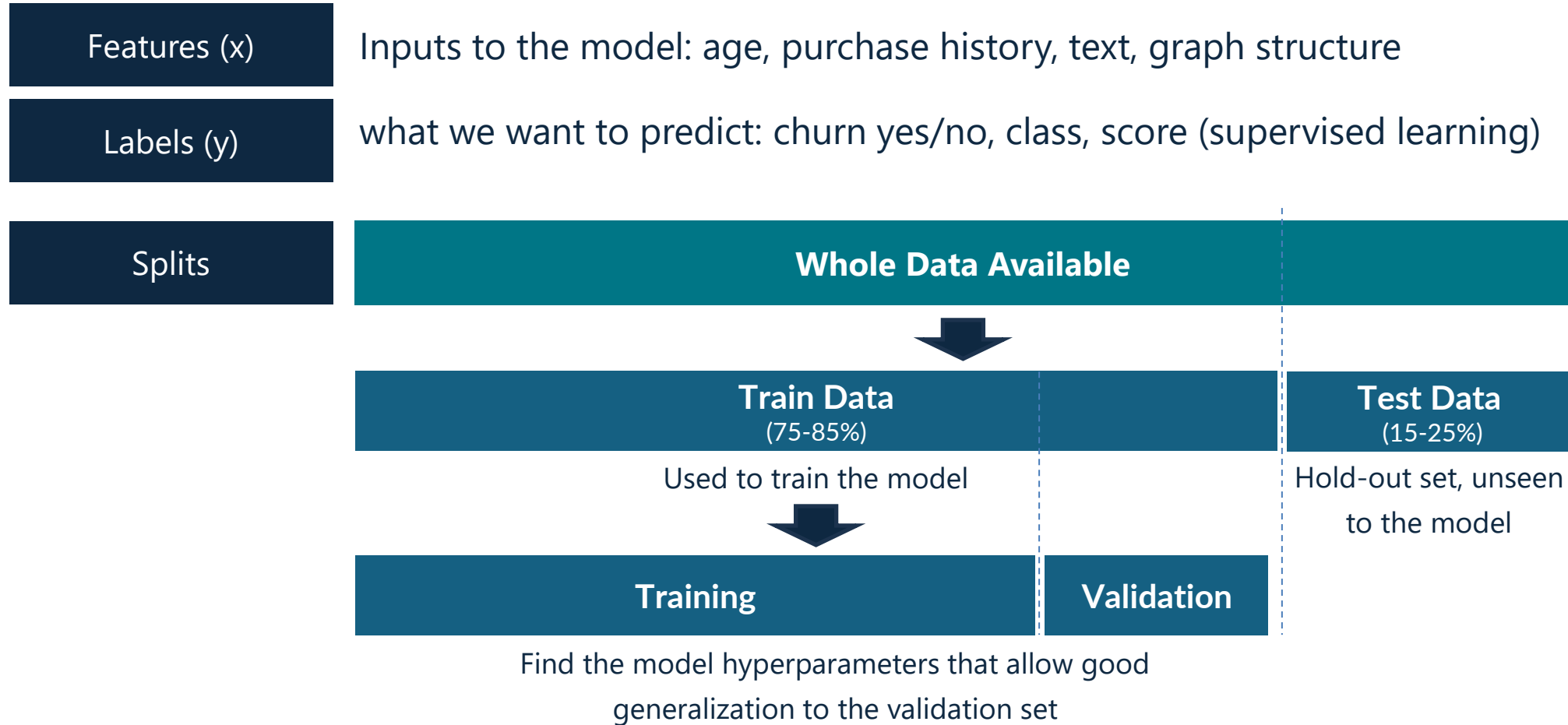
One update **for each sample** (chosen at random) in the dataset.

Minibatch GD



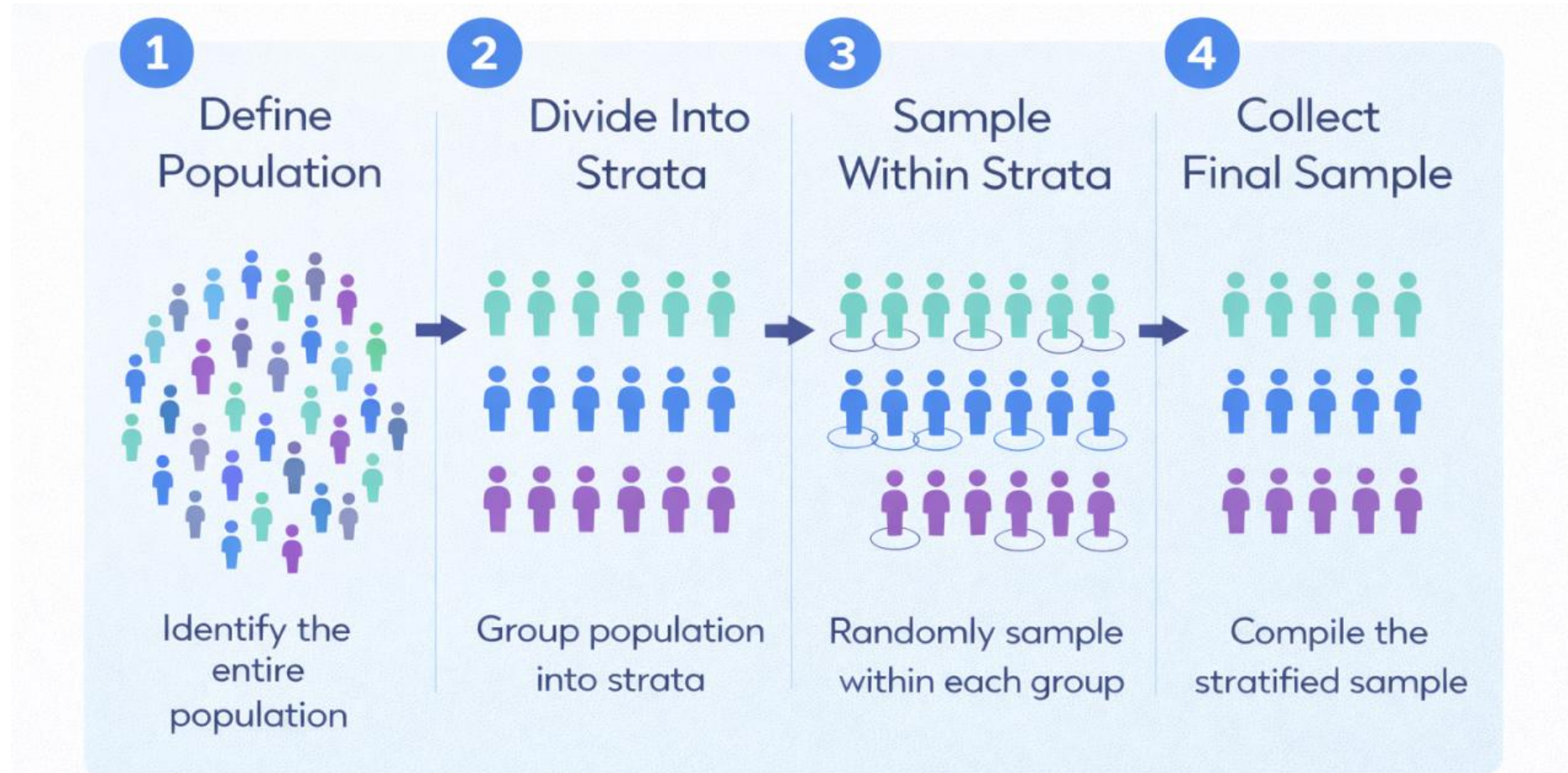
Split training data in **B chunks** (chosen randomly), compute update per chunk

Anatomy of a Dataset



How to do the Splits? Stratified Sampling

Respect original population distribution to generate the splits.

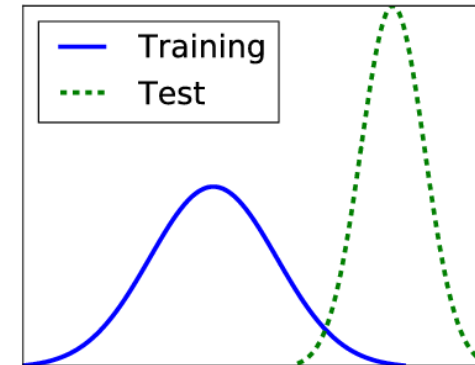


Data: Important Concepts

In-distribution
Out-of-distribution

Train and test/deployment data distribution might differ. AKA “**DATASET SHIFT**”

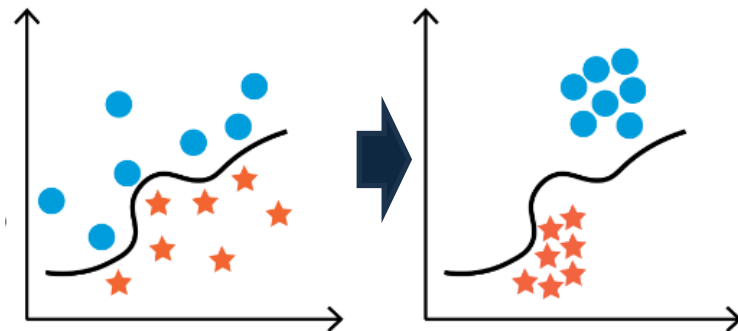
$$P_{train}(x, y) \neq P_{test}(x, y)$$



Data Drift

AKA “**COVARIATE SHIFT**”

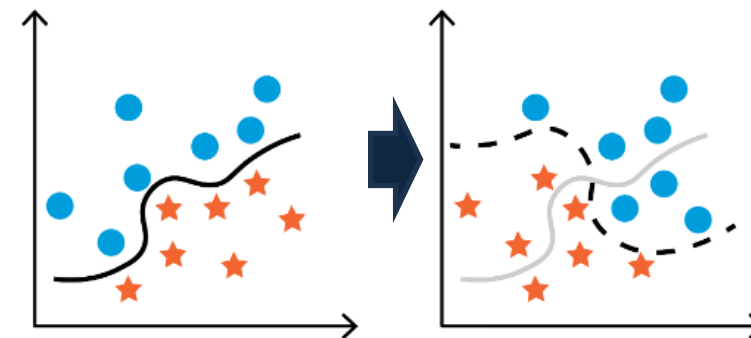
Change in the input distribution $P(x)$



You're seeing different inputs, but their meaning is unchanged

Concept Drift

Change in the output-input relationship $P(y|x)$.



Same input maps to different outputs.

Supervised Loss Functions

For a mini-batch of n samples:

Regression

Mean Squared Error (MSE)

$$\mathcal{L} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE)

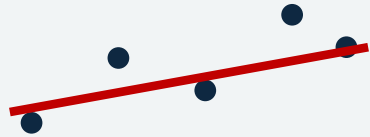
$$\mathcal{L} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$$

Classification

Cross-Entropy loss

$$\mathcal{L} = -\frac{1}{n} \sum_i \sum_c y_{i,c} \log \hat{y}_{i,c}$$

Underfitting vs Overfitting



Underfitting

Too simple. High train *and* val error.



Good fit

Captures the trend, ignores noise.



Overfitting

Memorizes noise. Fails on new data.

Underfitting vs Overfitting

Model

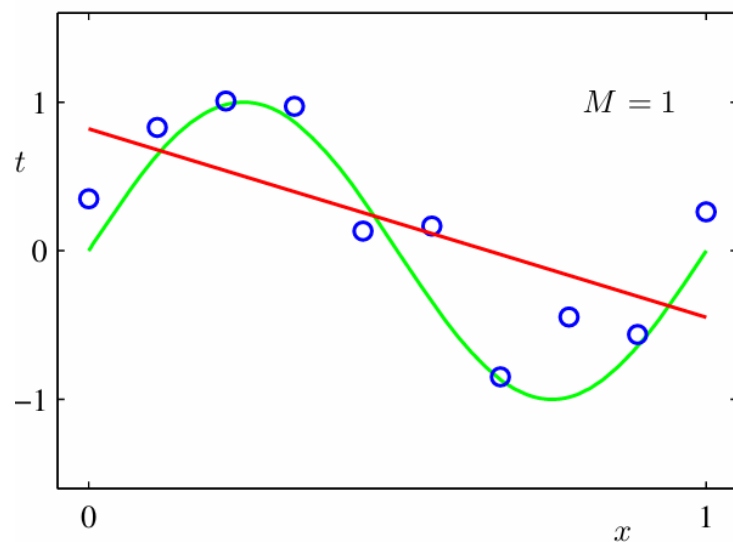
$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

Hyper-parametrized by M

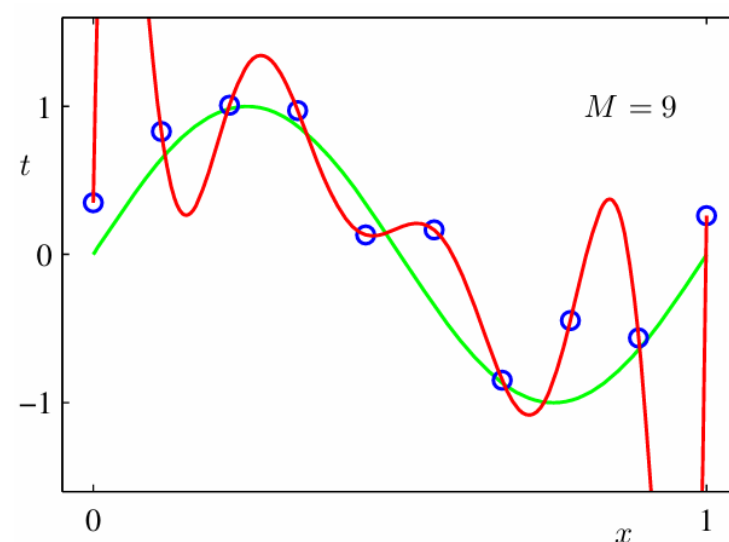
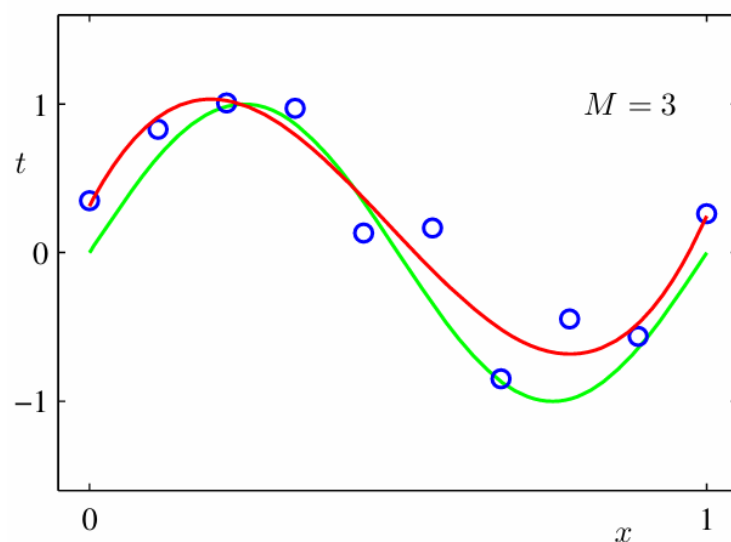
Loss

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$

MSE



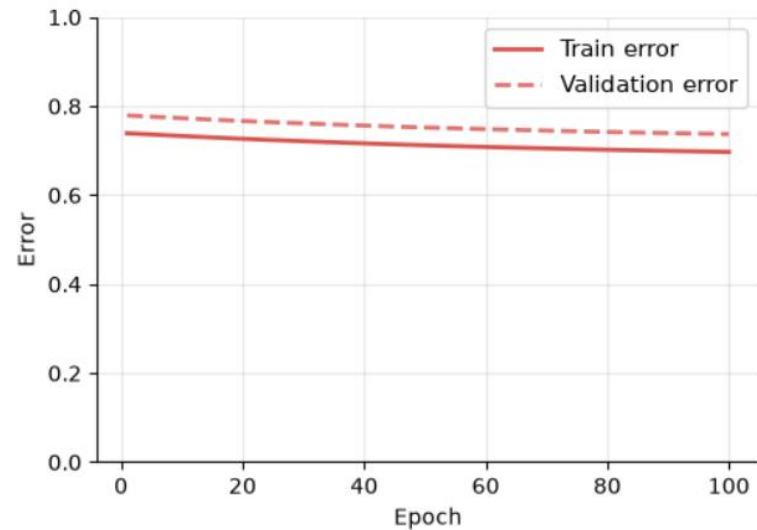
Underfitting



Overfitting

Underfitting vs Overfitting: Loss Curves

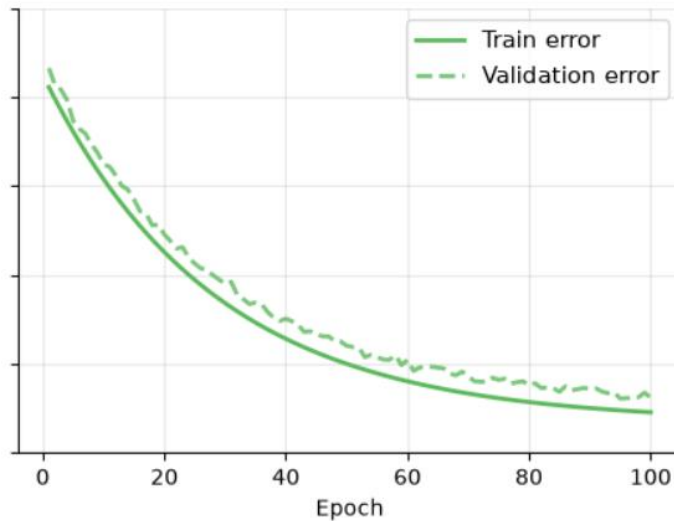
Underfitting



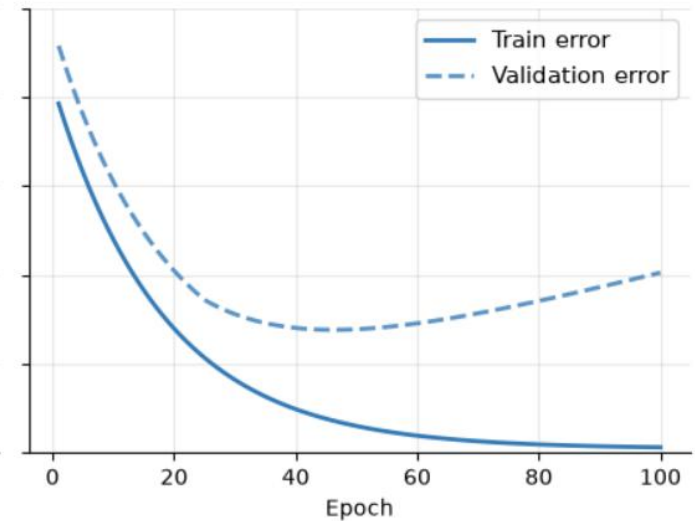
High Bias
(low variance)

- Increase model complexity
- Train longer

Good Fit



Overfitting



High Variance
(low bias)

- More training data
- Regularization

Loss Regularization

Intuition: large parameter magnitudes prone to cause overfitting

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{data}}(\theta)}_{\text{fit}} + \underbrace{\lambda}_{\text{Regularization Strength } (\geq 0)} \underbrace{R(\theta)}_{\text{complexity penalty}}$$

L2
(weight decay)

$$R(\theta) = \sum_j \theta_j^2 = \|\theta\|_2^2$$

Reduces magnitude of all weights (parameters) close to 0

L1
(Lasso)

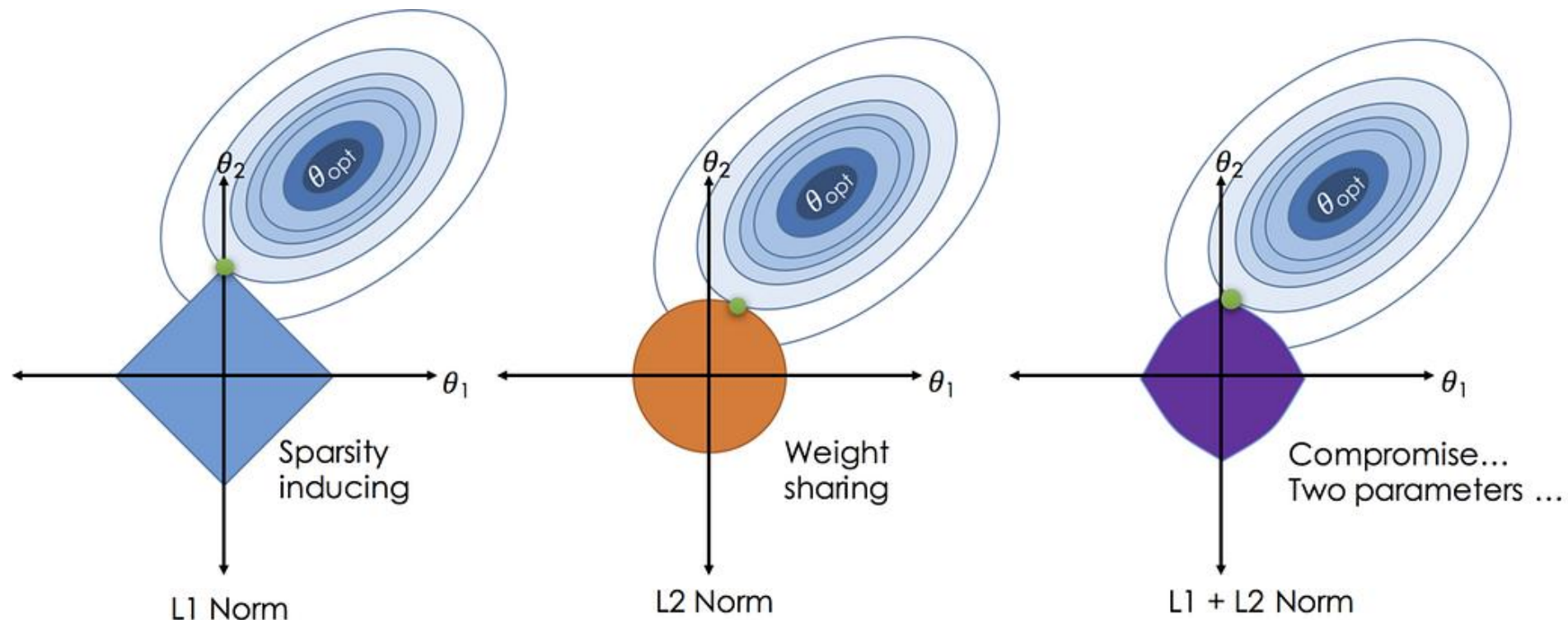
$$R(\theta) = \sum_j |\theta_j| = \|\theta\|_1$$

Induces sparsity → sets many weights to zero.

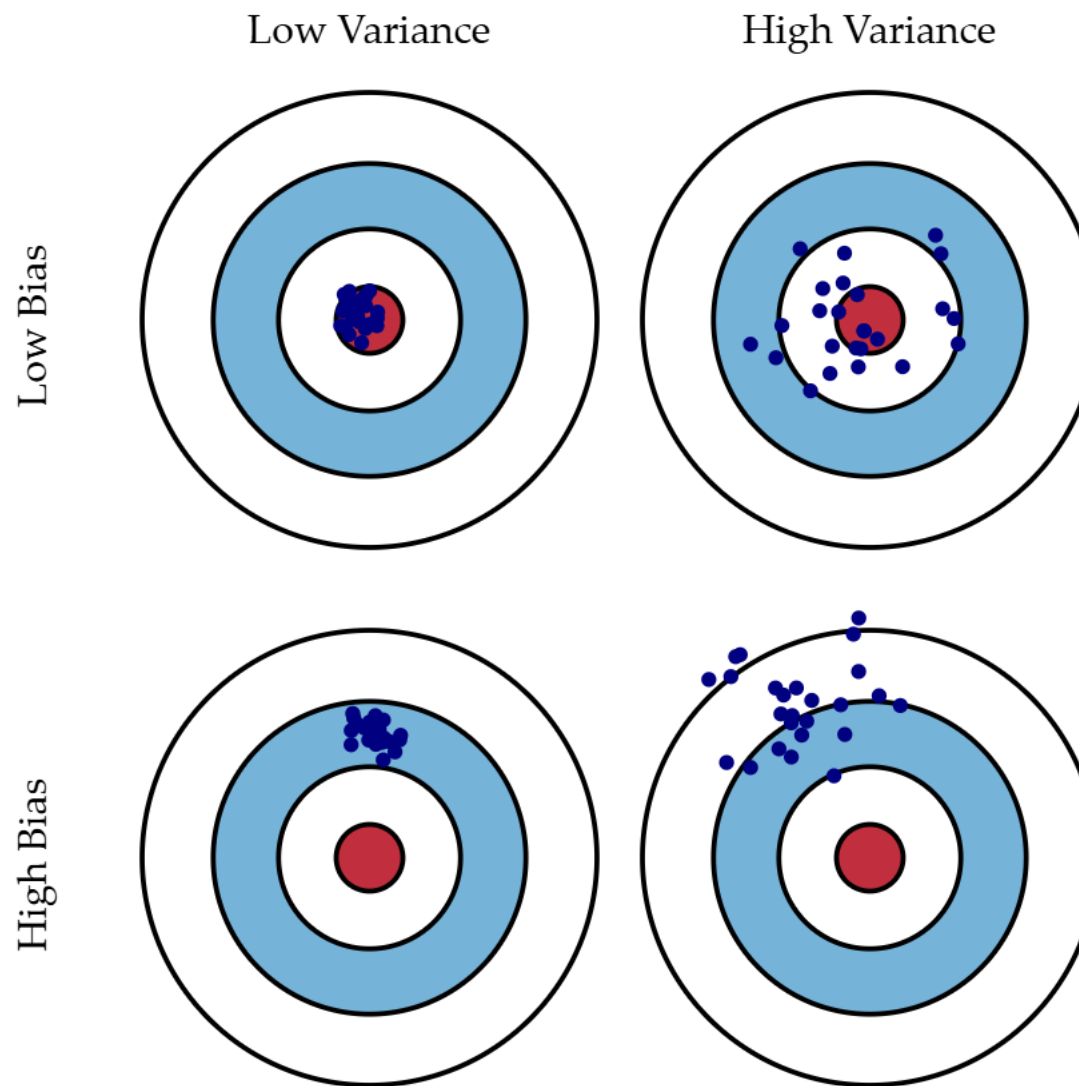
Elastic

$$R(\theta) = \alpha \|\theta\|_1 + (1 - \alpha) \|\theta\|_2^2$$

Loss Regularization

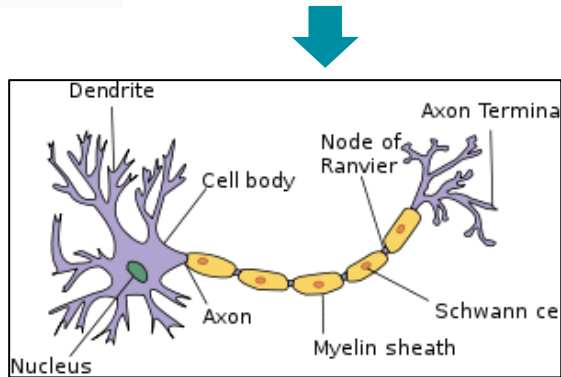
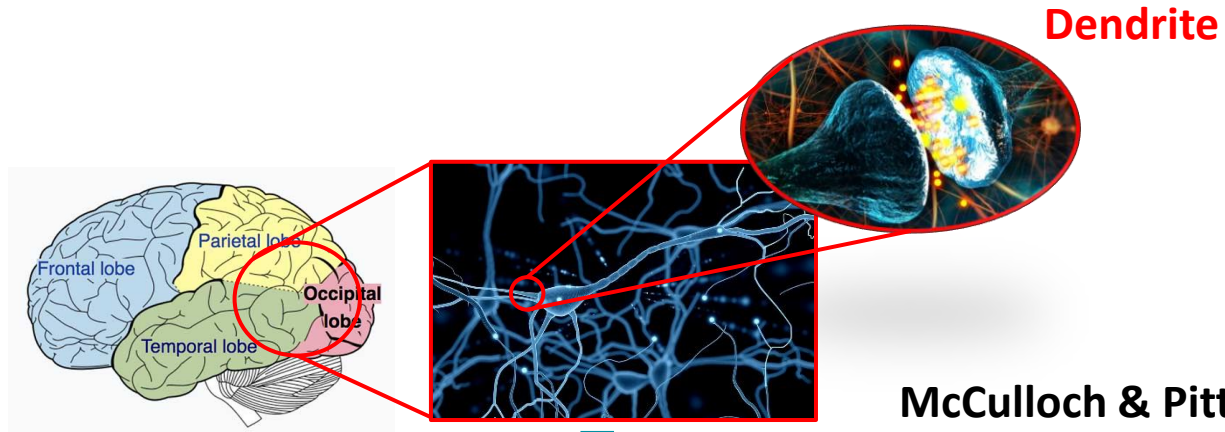


Bias-Variance Trade-off

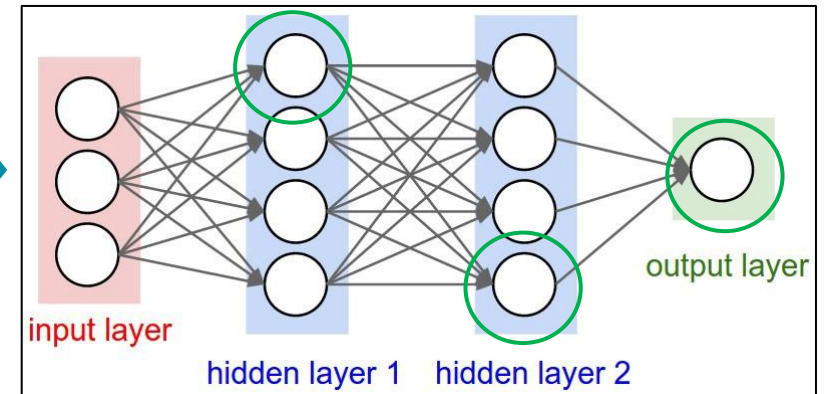
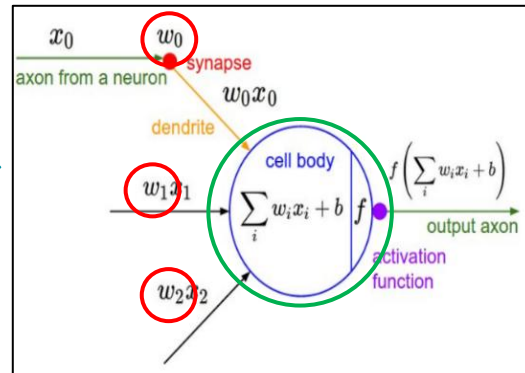


Deep Learning

Deep Learning

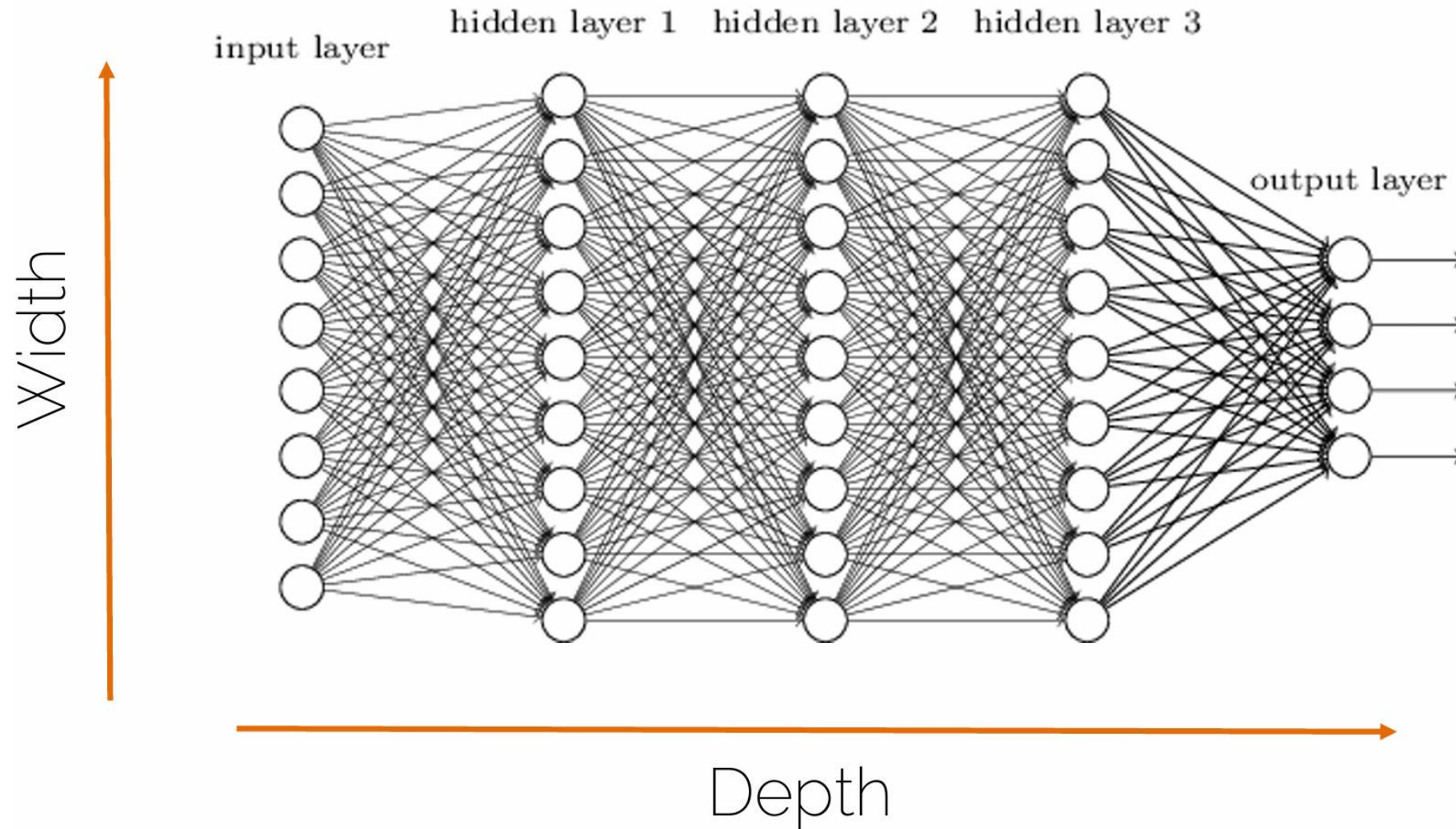


McCulloch & Pitts (1943).



Rosenblatt, F. (1958).

Deep Neural Network



Deep Neural Network

- Linear score function $f = Wx$
- Neural network is a nesting of 'functions'
 - 2-layers: $f = W_2 \max(0, W_1 x)$
 - 3-layers: $f = W_3 \max(0, W_2 \max(0, W_1 x))$
 - 4-layers: $f = W_4 \tanh(W_3, \max(0, W_2 \max(0, W_1 x)))$
 - 5-layers: $f = W_5 \sigma(W_4 \tanh(W_3, \max(0, W_2 \max(0, W_1 x))))$
 - ... up to hundreds of layers

Weight Matrices

Learnable parameters

Non-linear activations

Has no learnable parameters

Why do we need the non-linear activations?

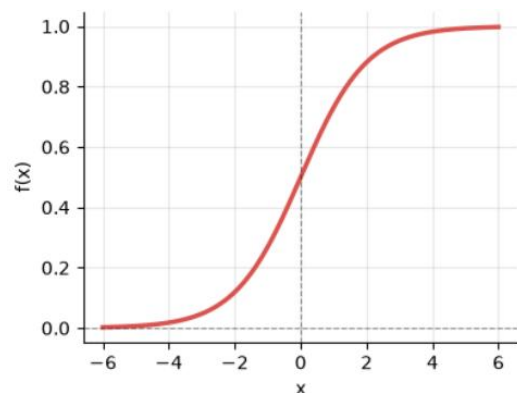
Without:

1-Layer: $h_1 = W_1x + b_1$

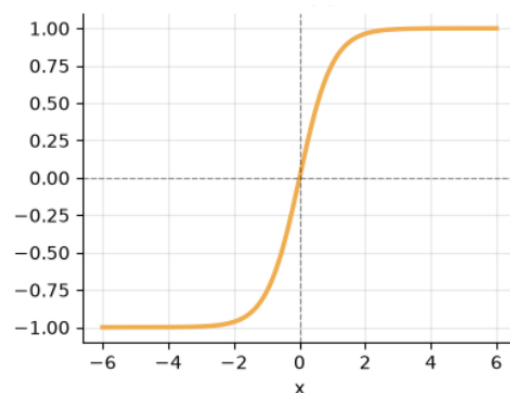
2-Layer: $h_2 = W_2h_1 + b_2 = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{W'}x + \underbrace{(W_2b_1 + b_2)}_{b'}$

Same as 1-layer

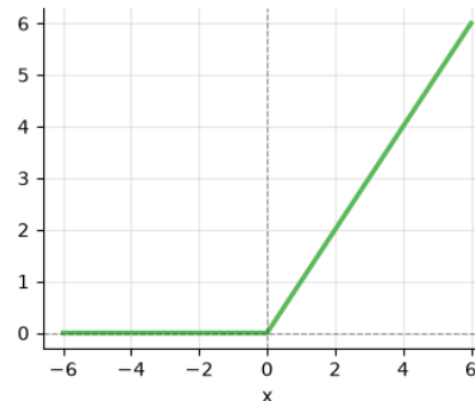
Sigmoid



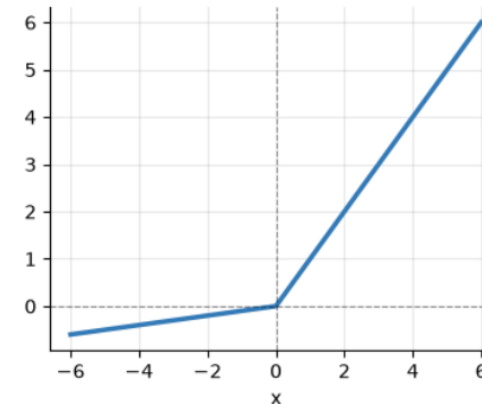
tanh



ReLU



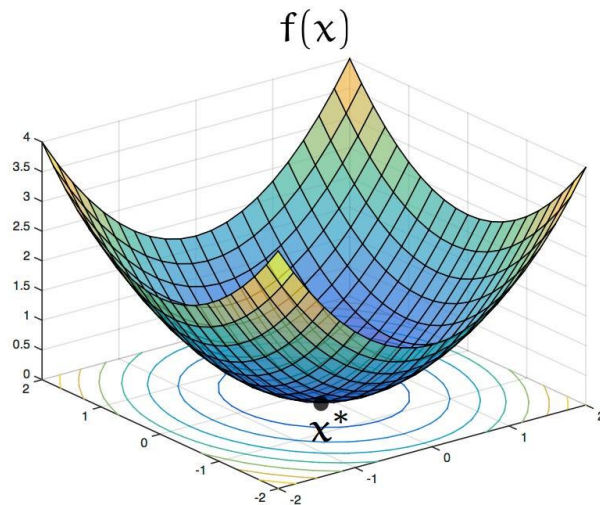
LeakyReLU



Deep Learning Optimization Landscape

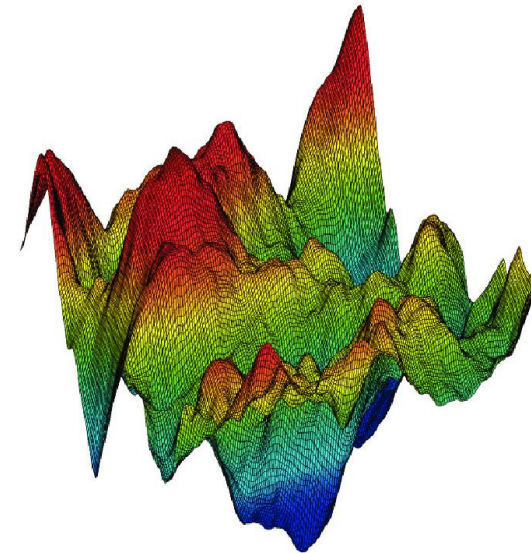
Convex Optimization

One **global** Minimum.



Non-Convex Optimization

Multiple local minima.



DEEP LEARNING

Images credit. Degen Code – Convex Optimization Introduction. AND. Sam Subey - Parameter estimation in stock assessment modelling: Caveats with gradient-based algorithms.

Training Loop

Forward pass

Feed data through the model → get prediction $\hat{y}$

Calculate loss/error

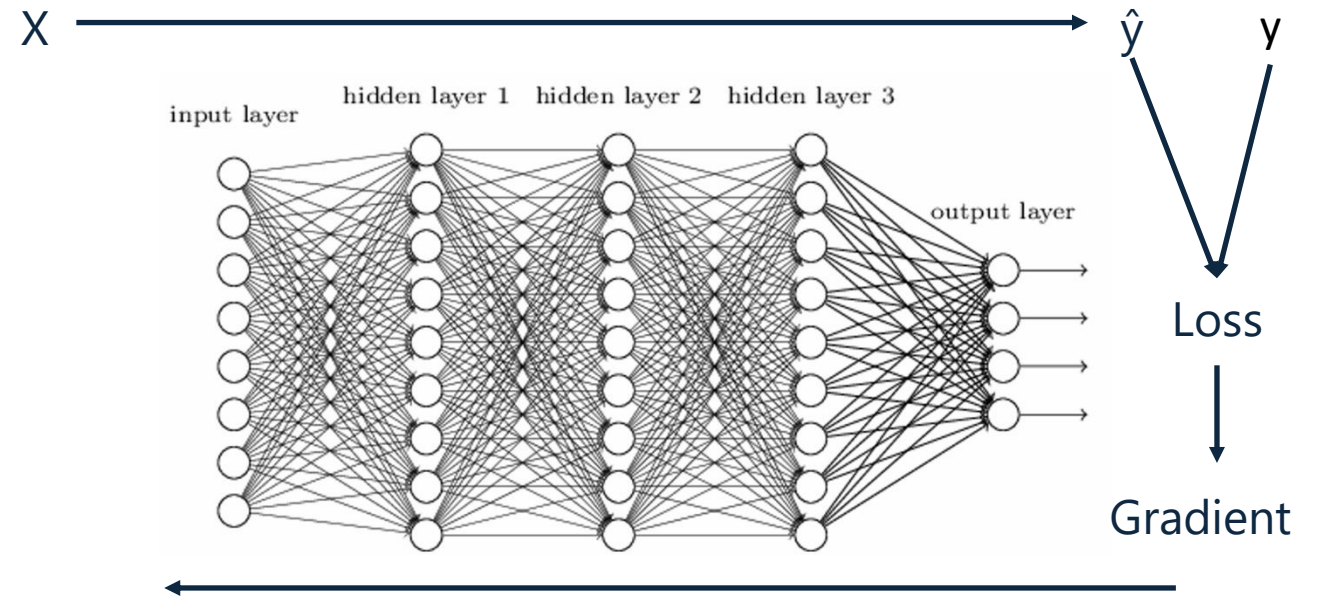
Measure how wrong $\hat{y}$ is relative to the true label y

Compute Gradient

Gradient: compute the direction of steepest error increase

Update Parameters

Move model parameters in the opposite direction



Repeat for all batches of data

Repeat for several epochs

PyTorch Training Loop

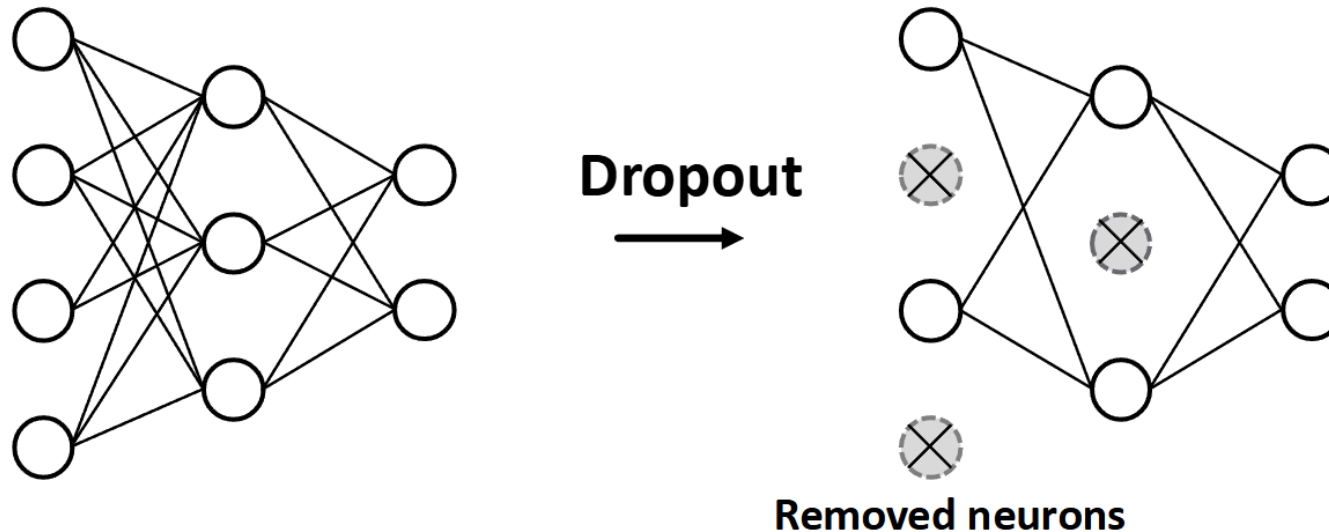
<pre>import torch import torch.nn as nn</pre>	Import PyTorch Library
<pre># --- 1. Toy dataset (y = 3x + noise) --- X = torch.randn(100, 1) y = 3 * X + 0.5 * torch.randn(100, 1)</pre>	Toy Dataset. X should be a Torch Tensor
<pre># --- 2. Model --- model = nn.Sequential(nn.Linear(1, 16), nn.ReLU(), nn.Linear(16, 1),)</pre>	Initialize model: 1-layer neural network with a ReLU activations function.
<pre># --- 3. Loss & optimizer --- criterion = nn.MSELoss() optimizer = torch.optim.Adam(model.parameters(), lr=1e-2)</pre>	Loss function and Optimizer
<pre># --- 4. Training loop --- for epoch in range(200): model.train() optimizer.zero_grad() preds = model(X) loss = criterion(preds, y) loss.backward() optimizer.step() if (epoch + 1) % 50 == 0: print(f"Epoch {epoch+1:3d} Loss: {loss.item():.4f}")</pre>	Train Loop

Test/Inference setup:
(no gradients needed)

```
model.eval()
with torch.no_grad():
    preds = model(X)
```


Dropout

- **Goal:** Regularize a neural net to prevent overfitting.
- **Idea:**
 - **During training:** with some probability p , randomly set neurons to zero (turn off)
 - **During testing:** Use all the neurons for computation



Metrics

Accuracy not Always the Right Classification Metric

99%

accuracy — catching zero fraud

If 99% of transactions are legitimate, “always predict *not fraud*” is 99% accurate, and **completely useless**.

False positive

innocent flagged → friction

False negative

fraud missed → financial loss

Ask: what is the cost of each type of mistake?

Confusion Matrix

	Predicted +	Predicted -
Actual +	TP true positive	FN Type II error
Actual -	FP Type I error	TN true negative

The diagonal is what you got right.

Everything else (precision, recall, F1) is derived from **these four numbers**.

Precision, Recall, F1-Score

Precision

$$TP / (TP + FP)$$

Of what I flagged, how much was right?

**When false alarms are costly.
E.g., spam filter**

Recall

$$TP / (TP + FN)$$

Of all real positives, how many caught?

**When a miss is costly.
E.g., cancer screening**

F1 Score

$$2 \cdot P \cdot R / (P + R)$$

Harmonic mean. Balances both.

When neither can be ignored

Choosing the right metric

PROBLEM	USE
Binary classification	Precision · Recall · F1 · AUC-ROC
Multi-class	F1 · confusion matrix · AUC-ROC
Regression	RMSE · MAE · R^2
Retrieval / RAG → Module II	Recall@k · Precision@k · MRR
Graph node classification → Module III	Accuracy per train / val / test split

Metrics

Regression

```
import torch

y_true = torch.tensor([3.0, -0.5, 2.0, 7.0])
y_pred = torch.tensor([2.5, 0.0, 2.0, 8.0])

rmse = torch.sqrt(torch.mean((y_true - y_pred) ** 2))
print(f"RMSE: {rmse.item():.4f}")
```

```
mae = (preds - y).abs().mean().item()
```

Classification

```
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Toy binary classification labels and predictions
y_true = np.array([1, 0, 1, 1, 0, 1, 0, 0, 1, 0])
y_pred = np.array([1, 0, 1, 0, 0, 1, 1, 0, 1, 0])

# --- With scikit-learn ---
print("=== scikit-learn ===")
print(f"Accuracy: {accuracy_score(y_true, y_pred):.4f}")
print(f"Precision: {precision_score(y_true, y_pred):.4f}")
print(f"Recall: {recall_score(y_true, y_pred):.4f}")
print(f"F1 Score: {f1_score(y_true, y_pred):.4f}")

# --- Manual ---
TP = ((y_pred == 1) & (y_true == 1)).sum()
TN = ((y_pred == 0) & (y_true == 0)).sum()
FP = ((y_pred == 1) & (y_true == 0)).sum()
FN = ((y_pred == 0) & (y_true == 1)).sum()

accuracy = (TP + TN) / len(y_true)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * (precision * recall) / (precision + recall)
```

Lab

Today's lab — your first model

- 1 Set up Python, a virtual environment & Jupyter
- 2 Load & explore a real tabular dataset
- 3 Train your first model — a decision tree
- 4 Read the metrics — precision, recall, F1

TOOLS

Python

pandas

numpy

scikit-learn

Jupyter

PyTorch

No GPU? Every lab has a **Google Colab** link.

Module I → Module II

WHAT WE COVERED

- The ML workflow: data → model → train → evaluate
- Metrics: precision, recall, F1, confusion matrix
- The attention intuition: learned relevance scores

WHAT'S NEXT

Large Language Models

How LLMs scale these ideas to billions of parameters — and why they still hallucinate.

Batch Normalization

- **Goal:** Stabilize neural networks training
- **Idea:** Given a batch of inputs (node embeddings)
 - Re-center the node embeddings into zero mean
 - Re-scale the variance into unit variance

Input: $\mathbf{X} \in \mathbb{R}^{N \times D}$
 N node embeddings

Trainable Parameters:
 $\gamma, \beta \in \mathbb{R}^D$

Output: $\mathbf{Y} \in \mathbb{R}^{N \times D}$
Normalized node embeddings

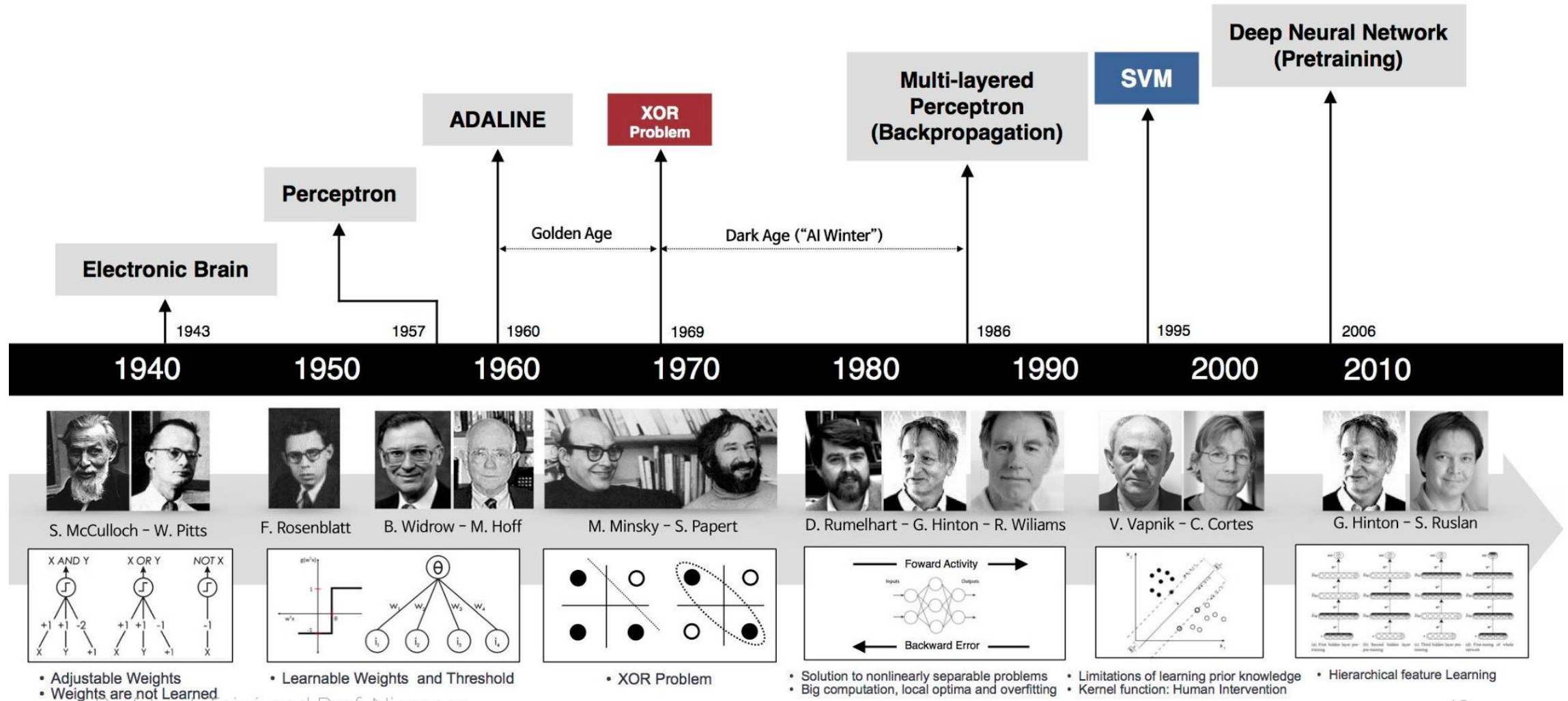
Step 1:
Compute the mean and variance over N embeddings

$$\mu_j = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_{i,j}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{i,j} - \mu_j)^2$$

Step 2:
Normalize the feature using computed mean and variance

$$\hat{\mathbf{x}}_{i,j} = \frac{\mathbf{x}_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$
$$\mathbf{y}_{i,j} = \gamma_j \hat{\mathbf{x}}_{i,j} + \beta_j$$

Brief History of AI



** Image credit: "Introduction to Deep Learning" Prof. Leal-Taixe, Prof. Niessner. TUM. 2017.

Dr. Daniel F. Pérez-Ramírez | LLMs & GNNs for Reasoning Beyond Text